INDEPENDENT TECHNICAL AUDIT

AForce OS

Codebase, security, scalability & production-readiness review

Scope: Expo mobile app · Express 5 API server · shared TypeScript libraries (PostgreSQL + Drizzle, Clerk, Stripe, ElevenLabs, OpenWeather)
Method: read-only code review + automated dependency, static-analysis (SAST) & privacy-dataflow scans · Generated May 31, 2026

Scorecard (1–10)

AREA	SCORE	ONE-LINE VERDICT
Architecture	8	Genuinely well-structured monorepo, contract-first, clear boundaries
Code Quality	8	Pure-function core, 50+ test files, consistent error handling
Security	6.5	Strong foundations, but one real IDOR + unpatched dependency CVEs
Scalability	6	Stateless auth is great; in-memory state blocks true multi-pod scale
Performance	7.5	Client-side scoring + caching + rate limits; well thought through
Production Readiness	5.5	Monitoring / error-tracking / analytics are stubs or absent
Technical Debt	7	Manageable — a few "god files" and legacy fallbacks
Investor Readiness	7	Solid engineering story; a short must-fix list stands between you and "enterprise-grade"

Overall: 7/10 — strong build, not yet hardened for a national launch. The architecture is better than the typical Replit app at this stage. The gaps are operational (monitoring, scale plumbing) and a small set of concrete security fixes — not structural rewrites.

What's genuinely strong

- **Modular, contract-first architecture.** OpenAPI spec (`lib/api-spec`) + generated Zod/React Query (`lib/api-zod`) enforce type safety across the client/server boundary. Schemas centralized in `lib/db` .
- **The "brain" is decoupled from the UI.** The scoring engine (`utils/scoringEngine.ts` , `depletionRate.ts`) is pure, dependency-free and unit-tested — runs client-side for zero-latency UI, with the server as source of truth. The right call.
- **Real test coverage** — 50+ test files in the mobile services layer.
- **Security fundamentals are present, not bolted on:** Clerk auth **fails closed** in production; every endpoint validates input with Zod; Drizzle parameterizes all queries (no SQL injection); Stripe webhooks verify signatures on the raw body; WebSocket connections authenticate via verified Clerk JWT (userId derived from token, not client input); CORS uses an allow-list in production.
- **Concurrency handled correctly** — `SELECT … FOR UPDATE` on intake writes; Postgres advisory locks serialize WHOOP token refreshes across replicas.

Prioritized Action Plan

CRITICAL Fix before scaling user signups

1. **IDOR in the Scans API.** `artifacts/api-server/src/routes/scans.ts` isolates data by a client-supplied `x-device-id` header instead of the authenticated `req.userId`. Anyone can read or write another user's scans by spoofing that header. Fix: authorize off the Clerk `userId`. The single most important item — it touches health-adjacent data.

HIGH Before national launch

2. **Patch dependency CVEs.** 0 critical, **15 high**, 14 moderate — but almost all are minor-version bumps. The 6 "high" Clerk packages are one advisory (`GHSA-w24r-5266-9c3c`) fixed by a patch release each; plus `lodash` , `js-cookie` , `fast-uri` , `path-to-regexp` , `brace-expansion` , `ip-address` . None require major upgrades.
3. **Wire a real Redis instance.** The cache, rate limiter and `redisClient` all silently fall back to **in-memory**. Rate limits and caches are therefore per-pod — they don't hold once more than one server instance runs.
4. **Make WebSocket broadcasts multi-pod.** `aforceHub.ts` keeps sockets in a local `Map` and only publishes to local connections. With >1 pod, a user on Pod A misses updates triggered on Pod B. Needs Redis Pub/Sub.
5. **Add error / crash tracking (e.g. Sentry)** on both server and mobile. Currently none exists — you are blind to production failures.

MEDIUM Fundraising-grade polish

6. **Real observability.** `metrics.ts` and `tracing.ts` are in-memory/console stubs. Wire to Prometheus / OpenTelemetry + log aggregation.
7. **Product analytics.** DAU / retention / streaks / conversion are **not instrumented**. For fundraising metrics, add one analytics layer (e.g. PostHog) — the highest-leverage *business* gap.
8. **SAST: 70 findings (69 medium, 1 low) — all low real-risk.** Mostly HTML-in-template-string in the Journal/Science HTML export reports (most interpolations already use `escapeHtml` ; close the remaining gaps) and a path-traversal flag in a *build script* (not runtime).
9. **Split the "god files":** `useAppStore.tsx` (~1400 lines), `routes/aforce.ts` (~1220), `scoringEngine.ts` (~1050). They work and are tested, but concentrate risk.
0. **Document a DB backup / restore strategy.** Nothing in code addresses it.

LOW Debt hygiene

1. Remove legacy `TODO(remove)` running-aggregate fallbacks in `scoringEngine.ts` ; reduce `as any` JSONB casts; harden advisory-lock key hashing against rare collisions.

Scalability — honest answer

LOAD	VERDICT
1,000 users	Comfortable today.
10,000 users	Fine once Redis is real (item 3) and CVEs are patched (item 2).
100,000 users	Needs items 3 + 4 (shared cache + WS Pub/Sub); watch the wide, frequently-updated `aforce_user_state` "god table" for row contention. No architectural rewrite required — it's plumbing.

Recommendation

The advice to also commission an independent senior engineer / fractional CTO review is sound. This report gives them a precise punch-list to validate against, which shortens — and cheapens — that engagement. The fastest high-value first batch is **#1 (IDOR fix)** + **#2 (dependency patches)**: both contained, low-risk, and the two items an auditor checks first.